

EXERCISE-3

Q1. Create a hierarchy of person, employee and developers.

```
JS q1.js > ...
1  function Person(name,age,address){
2      this.name=name;
3      this.age=age;
4      this.address=address;
5  }
6
7  function Employee(employeeId,competency){
8      this.employeeId=employeeId;
9      this.competency=competency;
10 }
11
12 function Developer(designation,salary){
13     this.designation=designation;
14     this.salary=salary;
15 }
16
17 Employee.prototype= new Person('Harsh',21,'Delhi');
18
19 Developer.prototype= new Employee('101','JS');
20
21 var Developer = new Developer('Software Developer', 150000);
22 console.log(Developer.name,Developer.age,Developer.address);
23 console.log(Developer.employeeId,Developer.competency)
24 console.log(Developer.designation,Developer.salary);
```

Output:

```
[Running] node "/home/harsh-raj/Bootcamp_Assignment/js_exercise3/q1.js"
Harsh 21 Delhi
101 JS
Software Developer 150000
```

Q2. Given an array, say [1,2,3,4,5]. Print each element of an array after 3 secs.

```
JS q2.js > ...
1  let arr = [1,2,3,4,5];
2
3  for(let i=0;i<arr.length;i++){
4      setTimeout(()=>{
5          console.log(arr[i]);
6      },3000*i);
7  }
```

Output:

```
[Running] node "/home/harsh-raj/Bootcamp_Assignment/js_exercise3/q2.js"
1
2
3
4
5

[Done] exited with code=0 in 12.143 seconds
```

Q3. Explain difference between Bind and Call (example).

Parameter	call()	bind()
Purpose	Invokes a function immediately	Return a new function with fixed this
Execution	Executes right away	Executes later
Return value	Returns the result of the function	Returns a new bound function
Arguments	Passed after the context, comma-separated	Can be passed now or later
Typical use	One time function call	Callbacks,event handlers,reuse

Example:

```
js q3.js > ...
1  function greet(greeting){
2    |    console.log(greeting,this.name);
3  }
4
5  const person1={name:"Abhishek"};
6  const person2={name:"Anil"};
7
8  greet.call(person1, "Hello");
9  greet.call(person2,"Hey");
10
11 //here bind fixes this to person1
12 const boundGreet=greet.bind(person1);
13
14 boundGreet("Hi");
15
16 /*now when call() tries to change this it can't because
17 bind() has higher priority and this is fixed t person1*/
18
19 boundGreet.call(person2,"yo");
```

Output:

```
[Running] node "/home/harsh-raj/Bootcamp_Assignment/js_exercise3/q3.js"
Hello Abhishek
Hey Abhishek
Hi Abhishek
yo Abhishek
```

Q4. Explain 3 properties of argument object.

Properties of argument object:

i) **argument.length**: This property returns the number of arguments passed to the function, this can be different from the number of declared parameters at the time of function definition.

Example:

```
JS q4.js > ...
1  function demo(a,b){
2      console.log(arguments.length);
3  }
4
5  demo(10,20,30);
```

Output:

```
[Running] node "/home/harsh-raj/Bootcamp_Assignment/js_exercise3/q4.js"
3
```

ii)indexed access(arguments[index]):This property allows each value that is passed to a function to be accessed using indexes(i.e,0,1,2,...). This property allows functions to handle a variable number of arguments dynamically, making them flexible and reusable.

Example:

```
8 //arguments[index]
9 function demo(){
10     for(let i=0;i<arguments.length;i++){
11         console.log(arguments[i]);
12     }
13 }
14
15 demo("JS","Html",1);
16 demo("React",2,3,4,5);
```

Output:

```
[Running] node "/home/harsh-raj/Bootcamp_Assignment/js_exercise3/q4.js"
JS
Html
1
React
2
3
4
5
```

iii)arguments.callee: This property holds a reference to the current executing function. This was mainly used for recursion in anonymous functions. It is now deprecated and its use is discouraged in modern Javascript, especially in strict mode.

Example:

```
//arguments.callee
function factorial(num){
  if(num<=1){
    return 1;
  }
  return num*arguments.callee(num-1);
}

console.log(factorial(5));
```

Output:

```
[Running] node "/home/harsh-raj/Bootcamp_Assignment/js_exercise3/q4.js"
120
```

Q5. Create a function which returns number of invocations and number of instances of a function.

```
JS q5.js > ...
1  function Tracker(){
2      if(this instanceof Tracker){
3          Tracker.instances++;
4      }else{
5          Tracker.invocations++;
6      }
7  }
8
9  Tracker.instances=0;
10 Tracker.invocations=0;
11
12 Tracker();
13 Tracker();
14 new Tracker();
15 new Tracker();
16 new Tracker();
17
18 console.log("Function calls:", Tracker.invocations);
19 console.log("Instances created:", Tracker.instances);
20
```

Output:

```
[Running] node "/home/harsh-raj/Bootcamp_Assignment/js_exercise3/q5.js"
Function calls: 2
Instances created: 3
```

Q6. Create a counter using closures.

```
JS q6.js > outer
1  function outer() {
2      let counter = 0;
3      return function () {
4          counter++;
5          return counter;
6      };
7  }
8  const increment = outer();
9  console.log(increment());
10 console.log(increment());
11 console.log(increment());
```

Output:

```
[Running] node "/home/harsh-raj/Bootcamp_Assignment/js_exercise3/q6.js"
1
2
3

[Done] exited with code=0 in 0.165 seconds
```


Q7. Explain 5 array methods with example.

1) map()

- Applies a function to every element
- Returns a new array
- Length of output = length of input

Example: `[1, 2, 3].map(x => x * 2); // → [2, 4, 6]`

2) filter()

- Keeps elements where callback returns true
- Returns a new array
- Length can be smaller

Example: `[1, 2, 3, 4].filter(x => x > 2); // → [3, 4]`

3) reduce()

- Accumulates values into a single result
- Can return anything (number, object, array)

Example: `[1, 2, 3].reduce((sum, x) => sum + x, 0); // → 6`

4) forEach()

- Executes a function for each element

Example: `[1, 2, 3].forEach(x => console.log(x));`

5) find()

- Returns the **first** element that matches
- Stops searching after first match

Example: `[1, 2, 3].find(x => x > 1); // → 2`

6) shift()

- Removes element from start of the array

Example: `[1,2,3].shift(); // [2,3]`

7) unshift()

- Adds element to the start of the array

Example: `[1,2,3].unshift(4); // [4,1,2,3]`

```

JS q7.js > ...
1  //Map
2  console.log("Map");
3  console.log([1, 2, 3].map(x => x * 2));
4
5  //Filter
6  console.log("Filter");
7  console.log([1, 2, 3, 4].filter(x => x > 2));
8
9  //Reduce
10 console.log("Reduce")           (parameter) x: number
11 console.log([1, 2, 3].reduce((sum, x) => sum + x, 0));
12
13 //forEach
14 console.log("forEach");
15 console.log([1, 2, 3].forEach(x => console.log(x)));
16
17 //Find
18 console.log("find");
19 console.log([1, 2, 3].find(x => x > 1));
20
21 //Shift
22 console.log("shift");
23 console.log([1,2,3].shift());
24
25 //Unshift
26 console.log("unshift");
27 console.log([1,2,3].unshift(4));

```

Output:

```

[Running] node "/home/harsh-raj/Bootcamp_Assignment/js_exercise3/q7.js"
Map
[ 2, 4, 6 ]
Filter
[ 3, 4 ]
Reduce
6
forEach
1
2
3
undefined
find
2
shift
1
unshift
4

```